MacZine
THE MACTECH SOLUTIONS NEWSLETTER
ISSUE Nº 062 · OCTOBER 2026 · MACTECHSOLUTIONSLLC.COM/MACZINE

PLATFORM SPOTLIGHT · TRUST CODEX

One Control, Followed From Log File to a Score a Prime Can Check

How Trust Codex turns one Sunday evidence run into an objective verdict, a recomputed SPRS methodology score, a hash-chained ledger entry and a signed proof.

BY **PATRICK CARUSO**

READING TIME · 7 MIN

TRUST-CODEX · CONTINUOUS-MONITORING · SPRS · ENCLAVEWATCH · AUDIT-LOGGING

Issue 52 made the case that a contractor's CMMC posture should be a continuous, tamper-evident record rather than a number affirmed once a year, on the same reasoning the DoD CIO applied when the Cyber Security Risk Management Construct replaced RMF snapshot authorization for the Department's own systems. That piece argued the principle. This one shows the plumbing, because a principle a vendor cannot demonstrate mechanism by mechanism is just a slogan with a diagram.

The clearest way to show it is to follow a single requirement through the machine. Take 3.3.1, create and retain system audit logs. It is worth 5 points under the DoD Assessment Methodology, it has six assessment objectives, and Issue 50 already walked through what an assessor asks for on each of them. Here is what happens to it inside Trust Codex between a Sunday collector run and a prime reading the result on a Monday.

FIELD COPY
ISSUE Nº 062
TRUST CODEX

5

POINTS 3.3.1 CARRIES UNDER THE DOD ASSESSMENT METHODOLOGY, DEDUCTED IN FULL IF ANY ONE OF ITS SIX OBJECTIVES IS NOT MET

7 days

DEFAULT FRESHNESS BUDGET ON AN AUTOMATED CHECK RESULT BEFORE IT STOPS COUNTING TOWARD A VERDICT

320

NIST SP 800-171A OBJECTIVES, EACH CITING ITS EVIDENCE BY HASH AND THE TIME IT WAS OBSERVED

Sunday: the evidence is collected and stays home

The run starts inside the CUI enclave. EnclaveWatch's OS collector writes the audit policy and its subcategories, the security and system event log configuration, and a set of hardening checks against the vault host. Two of those checks speak directly to 3.3.1: one confirms the required audit policy categories are present, the other that PowerShell logging is enabled. Azure collection supplies the third: whether the subscription's activity log is exported, and whether it is retained for at least 365 days.

None of those files leave the vault. What crosses the boundary to Codex is a manifest: each file's path, its SHA-256 hash, the time it was collected, and the directory on the vault host where the bytes are retained. Codex holds the fingerprint and the address, never the evidence. That is a design choice with a cost, since an assessor still retrieves the files on the vault under its own authentication, and it is the right one for a system whose whole job is to sit outside the CUI boundary.

On arrival: six objectives are adjudicated

When the bundle is accepted, Codex rescores in the background. It used to be otherwise. Until early October, technical evidence could sit unscored until the next weekly review triggered a pass; now every OS evidence bundle moves the adjudication on arrival.

The resolver treats evidence in two classes, and the distinction is the most important honesty decision in the engine. Only an automated check can produce a verdict on an objective. For 3.3.1, objective [a] is decided by the audit-category check, [c] by the PowerShell logging check, [f] by either of the two activity-log checks. All required checks passing and fresh means MET; a failing check, or one never observed, means NOT MET. A check result has a freshness budget, a week by default, so a pass from last month stops counting.

Everything else, the raw audit files, the audit configuration register with its 90-day review cadence, the Audit and Accountability Policy, is substantiation. It is cited, hashed, dated and shown to the assessor, and it never votes. Letting

a late register flip a verdict would cascade a single overdue entry into a failed control and an auto-drafted POA&M. Substantiation carries the honesty; verdicts stay on things a machine can actually test.

Then the requirement rolls up under the rule the CMMC Level 2 assessment guide states outright: one NOT MET objective fails the whole requirement. The only exceptions are the documented elevators the guide recognizes, such as inheritance from an external service provider or an enduring exception, each with its own paper trail. An earlier version of the engine once let a legacy "implemented" flag overrule two failing objectives on 3.5.3. The code now carries a comment explaining exactly how that happened, and the clamp that stops it.

The same pass: the score moves, and the old score stays

If 3.3.1's finding changes, two records are written. The first is a control history row with the prior and new findings, the prior and new objective verdicts, and the trigger that caused it, in this case a persisted validator run. The second is a new row in the SPRS score history, computed with the same Annex A arithmetic that produces the number a contractor types into SPRS. A failing 3.3.1 costs the full 5.

The engine never overwrites a score. It adds one, and the one it replaced stays in the chain where anyone holding a checkpoint can find it.

Each score row records the score, the per-control deltas, the trigger, and the hash of the row before it, and its own content hash covers all of that. The chain is linear per organization, enforced by unique indexes so two concurrent rescores cannot fork history. A weekly heartbeat row is written even when nothing moved, so a quiet week is an attested fact rather than an absence.

None of this puts a number into SPRS. SPRS is the Department's system. Codex computes the methodology score continuously; the contractor still posts it, and a senior official still affirms it.

◆ TWO SCORES, ONE AUTHORITY

Every history row carries two numbers. One comes from the older implementation-status lanes, the other from the objective-level adjudication described here. Each organization starts with the lanes as its published score. An administrator can switch authority to the adjudication engine only after reading every control where the two disagree, restating the count of disagreements, and justifying any that remain. The switch is itself written to the score history and the audit log, and it can be reversed. Running both in parallel is how you find out whether the new engine is right before you stake a number on it.

The ledger: why "tamper-evident" and not a stronger word

Both new rows become leaves in the organization's witness ledger, a Merkle tree in the style of Certificate Transparency, appended in the same database transaction so a history row and its leaf commit together or not at all. Database triggers refuse any edit or deletion of a row in the score history, the control history or the ledger itself for as long as the organization exists.

Triggers stop the application and a careless administrator. They do not stop someone with superuser rights to the database, which is why the word is tamper-evident and not immutable. The evidence has to come from outside the database, and that is the part that switches on when our signing key is issued. From then on, once a day, if any log has moved or a week has passed, Codex will sign a checkpoint with an Ed25519 key: one global Merkle root over every organization's log head, chained to the previous checkpoint. The statement reveals no tenant names and not even how many tenants exist. It will be committed to a public git repository whose workflow refuses any checkpoint that fails to chain to the last one or is signed by a key it does not know. Rewrite a past score after that and the root no longer matches a statement that anyone with a clone already holds.

Monday: a prime checks it without trusting us

Once the key is live, the contractor mints an attestation link and sends it. The prime opens it on mactechsolutionsllc.com/verify, and the page does the work in the prime's own browser: checks the document's signature against a pinned public key, walks the score history's hash chain, and confirms the organization's ledger head sits under a checkpoint in the public mirror. The document carries scores, per-control findings, freshness and proofs. It carries no CUI, no evidence and no objective text, and it names the company only if the company consented to being named on that link.

The page also says, in plain words, what a pass does not prove: that the evidence behind the findings is accurate. A hash proves the audit policy file is the one that was collected on Sunday. It cannot prove the file reflects the machine. That remains an assessor's job, and when assessments resume the same history goes into a signed binder, control by control and objective by objective, with a dependency-free `verify.mjs` that checks every file, signature and ledger proof offline and prints VERIFIED or names the check that failed.

That is the difference a continuous record makes to 3.3.1. The prime is no longer reading a claim that the audit logging works. It is reading when the checks last ran, what they found, what the score did about it, and proof that nobody went back and tidied up afterwards.

Correction, November 3, 2026: An earlier version of this article described daily signed checkpoints and live attestation links in the present tense. Both are built, and both start once MacTech's signing key is issued. Until then the history is hash-chained and protected against edits by the application, but it is not yet signed or publicly mirrored. ◆

MORE ISSUES
mactechsolutionsllc.com/maczine
new field copies with every merge

WORK WITH US
CMMC readiness · CUI enclaves · RMF
mactechsolutionsllc.com/contact

MACZINE
Written, reviewed and versioned in the open.
Published from Git, reviewed like code.